Cahier des charges — Back-office MoBis (Bus Maurice)

Version : 1.0
Date : 29 octobre 2026
Statut : Proposition validable
Périmètre : Partie administration et base de données associée
Source analysée : prototype `https://bus-maurice-voyageurs.sakabelo.chatgpt.site/admin`
Remarque : l'URL `/admin/objectif` renvoie actuellement une erreur 404 ; l'analyse porte donc sur le back-office `/admin` et sur les données de démonstration qu'il expose.

1. Contexte et objectifs

Le back-office MoBis permet à l'exploitant du réseau de bus de Maurice de consulter et de gérer les données du réseau : bus, arrêts, trajets, zones, personnel, utilisateurs, signalements, avis et points.

Le prototype actuel est une interface statique de démonstration. Les données sont stockées dans le `localStorage` du navigateur (`mobis-admin-v1`), les rôles sont simulés et aucune authentification réelle n'est appliquée.

L'objectif de ce cahier des charges est de définir la version administrable réelle :

- avec une authentification et une gestion des droits ;
- avec une persistance en base de données relationnelle ;
- avec des règles métier et des contrôles serveur ;
- avec des critères d'acceptation testables ;
- avec des exigences de sécurité, de performance et d'accessibilité.

Objectifs mesurables

Objectif	Indicateur cible
Centraliser la gestion du réseau	100 % des entités du prototype persistées en base
Sécuriser l'accès administrateur	0 accès non authentifié aux routes `/admin` et aux API d'administration
Fiabiliser les données	Contraintes d'unicité, de clés étrangères et de validation serveur
Tracer les actions sensibles	100 % des créations, modifications, suppressions, modérations et ajustements audités
Rendre l'interface utilisable	Accessibilité WCAG 2.2 AA, responsive 360–1920 px, FR/EN
Garantir la qualité	Tests unitaires, intégration, E2E Playwright et sécurité au vert

2. Périmètre

2.1 Inclus

- Authentification des comptes administrateur, opérateur et modérateur.
- Tableau de bord et indicateurs du réseau.
- Gestion des bus.
- Gestion des arrêts.
- Gestion des trajets et de leur ordre d'arrêts.
- Gestion des zones géographiques.
- Gestion du personnel administratif.
- Gestion des comptes voyageurs.
- Traitement des signalements.
- Modération des avis.
- Ajustement manuel des points.
- Recherche, filtres, pagination et export CSV.
- Journal d'audit.
- Internationalisation FR/EN et thème clair/sombre.
- Base de données, migrations, jeux de données et sauvegardes.

2.2 Exclu de cette version

- Application mobile native.
- Géolocalisation temps réel des bus.
- Calcul d'itinéraires en temps réel.

- Paiement en ligne.
- Messagerie temps réel.
- Notifications push.
- Barème automatique des points.
- Gestion des chauffeurs et des plannings.
- Intégration avec un fournisseur cartographique réel.
- Application voyageur complète.

2.3 Hypothèses

- Le back-office est servi sous `/admin`.
- Le back-office partage la même base que l'interface voyageur, avec des rôles distincts.
- La langue principale est le français ; l'anglais est une traduction secondaire.
- La devise est la roupie mauricienne (MUR) et le fuseau de référence est UTC.
- Les données de démonstration restent fictives.
- La cible technique recommandée est PHP 8.2 + MySQL 8.0, cohérente avec un hébergement mutualisé ou un serveur Linux classique.

3. Acteurs et rôles

Rôle	Description	Accès au back-office
`traveller`	Voyageur utilisateur de l'interface publique	Non
`operator`	Agent d'exploitation du réseau	Oui, périmètre opérationnel
`moderator`	Modérateur des contenus et signalements	Oui, périmètre modération
`admin`	Administrateur complet	Oui, tous les droits
Super-administrateur	Option cible pour les opérations sensibles	Oui, droits irrévocables contrôlés

Matrice des permissions

Permission	Opérateur	Modérateur	Administrateur
`dashboard.read`	Oui	Oui	Oui
`buses.read`	Oui	Oui	Oui
`buses.write`	Oui	Non	Oui
`stops.read`	Oui	Oui	Oui
`stops.write`	Oui	Non	Oui
`routes.read`	Oui	Oui	Oui
`routes.write`	Oui	Non	Oui
`zones.read`	Oui	Non	Oui
`zones.write`	Oui	Non	Oui
`users.read`	Non	Oui	Oui
`users.write`	Non	Non	Oui
`staff.read`	Non	Non	Oui
`staff.write`	Non	Non	Oui
`reports.read`	Oui	Oui	Oui
`reports.review`	Oui	Oui	Oui
`reviews.read`	Oui	Oui	Oui
`reviews.moderate`	Non	Oui	Oui
`points.read`	Non	Non	Oui
`points.adjust`	Non	Non	Oui
`exports.run`	Oui	Oui	Oui
`audit.read`	Non	Non	Oui

Permission	Opérateur	Modérateur	Administrateur
`settings.write`	Non	Non	Oui

4. Besoins fonctionnels

4.1 Synthèse

ID	Module	Besoin	Priorité
BF-ADM-01	Authentification	Connexion, déconnexion, sessions et rôles	Must
BF-ADM-02	Tableau de bord	Indicateurs et activité récente	Must
BF-ADM-03	Bus	CRUD et états des bus	Must
BF-ADM-04	Arrêts	CRUD et coordonnées	Must
BF-ADM-05	Trajets	CRUD et arrêts ordonnés	Must
BF-ADM-06	Zones	CRUD et polygones	Must
BF-ADM-07	Personnel	CRUD des comptes staff	Must
BF-ADM-08	Utilisateurs	Consultation et suspension	Must
BF-ADM-09	Signalements	Traitement des signalements	Must
BF-ADM-10	Avis	Modération des avis	Must
BF-ADM-11	Points	Ajustement manuel et historique	Must
BF-ADM-12	Export	Export CSV filtré	Should
BF-ADM-13	Recherche	Recherche, filtres et pagination	Must
BF-ADM-14	Audit	Journal des actions sensibles	Must
BF-ADM-15	Interface	FR/EN, thème clair/sombre	Should
BF-ADM-16	Sécurité	CSRF, sessions, limitation des tentatives	Must

4.2 Détail par module

BF-ADM-01 — Authentification et autorisation

Description : un compte staff doit pouvoir se connecter au back-office et accéder uniquement aux modules autorisés par son rôle.

Règles :

- Connexion par e-mail et mot de passe.
- Mot de passe haché avec `password_hash()` en Argon2id ou bcrypt.
- Session côté serveur avec cookie `HttpOnly`, `Secure`, `SameSite=Lax`.
- Jeton CSRF obligatoire pour toute requête d'écriture.
- Expiration après 30 minutes d'inactivité et 8 heures maximum.
- Verrouillage temporaire après 5 échecs consécutifs sur 15 minutes.
- Message d'erreur générique en cas d'échec, sans révéler si l'e-mail existe.
- MFA obligatoire pour le rôle `admin` en production.
- Déconnexion explicite et révocation de la session.
- Toute tentative de connexion, réussie ou non, est journalisée.

Critères d'acceptation :

- Un compte `operator` ne peut pas ouvrir `/admin/staff`, `/admin/users` en écriture, `/admin/points` ni `/admin/audit`.
- Un utilisateur non connecté est redirigé vers `/admin/login`.
- Une requête API sans CSRF retourne `403`.
- Après 5 échecs, le compte est verrouillé 15 minutes.

- La session est détruite après déconnexion.

BF-ADM-02 — Tableau de bord

Description : afficher une vue synthétique du réseau.

Contenu obligatoire :

- Nombre de bus en service / total.
- Nombre de zones actives / total.
- Nombre d'utilisateurs.
- Nombre de signalements en attente.
- Nombre d'avis en attente de modération.
- Les 3 derniers signalements en attente.
- Un tableau des 4 premiers bus.
- Un accès direct aux modules concernés.

Critères d'acceptation :

- Les compteurs correspondent aux données de la base au moment du chargement.
- Les liens « Consulter » ouvrent le module avec le bon filtre.
- Un opérateur ne voit pas les données réservées à l'administrateur.
- Le tableau de bord ne révèle aucune donnée personnelle sensible.

BF-ADM-03 — Gestion des bus

Description : gérer la flotte de bus.

Champs :

- Code public (`B001`, etc.).
- Plaque d'immatriculation unique.
- Ligne.
- Direction.
- État : `active`, `inactive`, `breakdown`.
- Capacité optionnelle.

Règles :

- La plaque est unique, insensible à la casse.
- La plaque et la ligne sont obligatoires.
- Une suppression est journalisée.
- Un bus lié à des signalements ne doit pas être supprimé sans règle explicite ; par défaut, les signalements sont conservés avec `bus_id = NULL`.
- Le changement d'état est immédiat et visible dans le tableau de bord.

Critères d'acceptation :

- La création avec une plaque déjà existante retourne une erreur `409`.
- La modification d'un bus met à jour `updated_at` et crée une entrée d'audit.
- Le filtre par état retourne uniquement les bus correspondants.
- Un modérateur ne peut pas modifier un bus.

BF-ADM-04 — Gestion des arrêts

Description : gérer les arrêts du réseau.

Champs :

- Code public (`A001`, etc.).
- Nom.
- Latitude.
- Longitude.
- État : `active`, `inactive`.

Règles :

- Latitude comprise entre -90 et 90.
- Longitude comprise entre -180 et 180.
- Nom unique par couple latitude/longitude.
- Un arrêt utilisé par un trajet ne peut pas être supprimé (`RESTRICT`).
- Un arrêt inactif ne peut pas être ajouté à un nouveau trajet.

Critères d'acceptation :

- Une latitude de 100 est refusée avec un message explicite.
- La suppression d'un arrêt utilisé retourne `409`.
- Un arrêt inactif est exclu de la liste de sélection des trajets.
- Les coordonnées sont affichées avec 7 décimales maximum.

BF-ADM-05 — Gestion des trajets

Description : gérer les trajets et l'ordre des arrêts.

Champs :

- Code public (`T001`, etc.).
- Nom.
- Ligne.
- État : `active`, `inactive`.
- Liste ordonnée d'arrêts.

Règles :

- Un trajet contient au moins 2 arrêts : départ et terminus.
- L'ordre est unique et stocké dans `route_stops.stop_order`.
- Un même arrêt ne peut pas apparaître deux fois dans le même trajet.
- Les arrêts doivent être actifs au moment de l'enregistrement.
- La suppression d'un trajet supprime ses associations d'arrêts, mais pas les arrêts eux-mêmes.

Critères d'acceptation :

- Un trajet avec un seul arrêt est refusé.
- Le déplacement d'un arrêt met à jour l'ordre affiché et l'ordre en base.
- La suppression d'un trajet ne supprime aucun arrêt.
- Le détail d'un trajet affiche les arrêts dans l'ordre.

BF-ADM-06 — Gestion des zones

Description : gérer les zones géographiques du réseau.

Champs :

- Code public (`Z001`, etc.).
- Nom.
- État : `active`, `inactive`.
- Polygone : liste de points `[x, y]` ou géométrie.

Règles :

- Un polygone contient au moins 3 sommets.
- Les points doivent être dans les limites de la carte ou dans les limites géographiques définies.
- Le polygone ne doit pas être vide.
- En cible cartographique réelle, le stockage doit être compatible `POLYGON SRID 4326`.
- La suppression est journalisée.

Critères d'acceptation :

- Une zone avec 2 sommets retourne une erreur.
- L'ajout de 3 sommets permet l'enregistrement.
- Une zone inactive n'apparaît pas sur la carte publique.
- La modification d'une zone est auditée avec l'ancienne et la nouvelle valeur.

BF-ADM-07 — Gestion du personnel

Description : gérer les comptes du personnel administratif.

Champs :

- Code public (`P001`, etc.).
- Prénom, nom.
- E-mail unique.
- Rôle : `operator`, `moderator`, `admin`.
- État : `active`, `inactive`.

Règles :

- Un administrateur ne peut pas se retirer lui-même le rôle `admin`.
- Le dernier administrateur actif ne peut pas être désactivé.
- Un compte staff ne peut pas être supprimé s'il est l'auteur d'actions d'audit ; il est désactivé à la place.
- La réinitialisation de mot de passe est journalisée.
- Un opérateur ne peut pas gérer le personnel.

Critères d'acceptation :

- La désactivation du dernier admin est refusée.
- Un admin peut créer un opérateur.
- Un opérateur reçoit `403` sur l'API du personnel.
- Le changement de rôle est visible dans le journal d'audit.

BF-ADM-08 — Gestion des utilisateurs voyageurs

Description : consulter et modérer les comptes voyageurs.

Champs :

- Code public (`U001`, etc.).
- Prénom, nom.
- E-mail.
- Rôle.
- État.
- Solde de points.

Règles :

- L'e-mail est unique.
- Le mot de passe n'est jamais affiché.
- La suspension d'un compte révoque ses sessions actives.
- La réactivation est possible.
- Le solde de points est en lecture seule dans ce module ; il est modifié uniquement par le module Points.
- La suppression définitive est une opération sensible soumise à confirmation et à conservation légale.

Critères d'acceptation :

- La suspension retire l'accès à l'interface voyageur.
- La réactivation remet le statut à `active`.
- Un modérateur peut consulter la fiche mais pas modifier le statut.
- Aucune donnée de mot de passe n'est exposée par l'API.

BF-ADM-09 — Signalements

Description : traiter les signalements envoyés par les voyageurs.

Champs :

- Référence (`S001`, etc.).
- Bus concerné.
- Auteur.
- Type : `breakdown`, `other`.
- Commentaire.
- Statut : `pending`, `confirmed`, `dismissed`.
- Traité par, date de traitement.

Règles :

- Un signalement `pending` peut passer à `confirmed` ou `dismissed`.
- Confirmer un signalement ne modifie pas automatiquement l'état du bus.
- Le commentaire est affiché en texte échappé.
- Un signalement n'est pas supprimé ; il est conservé pour l'historique.
- Toute modération est auditée.

Critères d'acceptation :

- Le passage à `confirmed` enregistre `handled_by` et `handled_at`.
- L'état du bus reste inchangé après confirmation.
- Un signalement `dismissed` n'apparaît plus dans les éléments à traiter.
- Le filtre par statut fonctionne.

BF-ADM-10 — Avis

Description : modérer les avis des voyageurs.

Champs :

- Auteur.
- Trajet et/ou bus.
- Commentaire.
- Statut : `pending`, `published`, `hidden`.
- Modéré par, date de modération.

Règles :

- Un avis `pending` peut être publié ou masqué.
- Un avis masqué n'apparaît pas dans l'interface publique.
- Le contenu est échappé avant affichage.
- La modération est auditée.
- L'auteur ne peut pas modérer son propre avis.

Critères d'acceptation :

- Un avis publié est visible côté voyageur.
- Un avis masqué n'est plus visible côté voyageur.
- La modération enregistre le modérateur et la date.
- Un modérateur ne peut pas publier un avis qu'il a lui-même écrit.

BF-ADM-11 — Points

Description : ajuster manuellement les points des utilisateurs.

Champs :

- Utilisateur.
- Variation entière comprise entre -10 000 et +10 000.
- Motif obligatoire (3 à 250 caractères).
- Solde après opération.
- Auteur de l'ajustement.
- Date.

Règles :

- La variation ne peut pas être nulle.
- Le solde ne peut pas devenir négatif.
- L'opération est atomique : mise à jour du solde et insertion dans l'historique.
- Une clé d'idempotence empêche le double enregistrement en cas de double clic.
- Le barème automatique des points est hors périmètre.
- L'historique est conservé et non modifiable.

Critères d'acceptation :

- Un ajustement de +10 sur un solde de 5 donne 15.
- Un ajustement qui rendrait le solde négatif est refusé.
- Un motif vide est refusé.
- Un double envoi avec la même clé d'idempotence ne crée qu'une seule ligne.
- L'historique affiche l'ancien solde, la variation, le nouveau solde et l'auteur.

BF-ADM-12 — Export CSV

Description : exporter les données du module courant.

Règles :

- Export du jeu filtré courant.
- Encodage UTF-8 avec BOM.
- Séparateur `;`.
- Échappement des guillemets.
- Protection contre l'injection de formules (`=`, `+`, `-`, `@`).
- Nom de fichier `mobis-<module>-<date>.csv`.
- Export limité à 10 000 lignes en synchrone ; au-delà, génération asynchrone.
- L'export est journalisé.

Critères d'acceptation :

- Le fichier s'ouvre correctement dans Excel avec les accents.
- Une valeur commençant par `=` est préfixée par une apostrophe.
- Le nombre de lignes correspond au filtre actif.
- L'export ne contient que les colonnes autorisées pour le rôle.

BF-ADM-13 — Recherche, filtres et pagination

Description : retrouver rapidement une entrée.

Règles :

- Recherche serveur sur les champs principaux.
- Filtre par statut.
- Pagination de 20 lignes par défaut.
- Tri stable et documenté.
- Conservation des filtres dans l'URL.
- État vide explicite.

Critères d'acceptation :

- La recherche « Rose » retourne les bus, arrêts et trajets concernés.
- Le filtre `breakdown` ne retourne que les bus en panne.
- La pagination ne perd pas le filtre.
- Le rechargement de l'URL conserve la recherche.

BF-ADM-14 — Journal d'audit

Description : tracer les actions sensibles.

Événements à journaliser :

- Connexion réussie et échouée.
- Déconnexion.
- Création, modification et suppression d'une entité.
- Changement de rôle ou de statut.
- Modération d'un signalement ou d'un avis.
- Ajustement de points.
- Export CSV.
- Modification des paramètres.

Règles :

- Le journal est en lecture seule via l'interface.
- Il contient l'acteur, l'action, l'entité, l'identifiant, l'ancienne valeur, la nouvelle valeur, l'IP, le user-agent et la date.
- Conservation minimale : 12 mois.
- Aucune donnée de mot de passe ni de jeton n'est journalisée.

Critères d'acceptation :

- Chaque modification d'un bus crée une ligne d'audit.
- Une tentative de connexion échouée est visible dans l'audit.
- Un opérateur ne peut pas consulter le journal.
- Le journal ne peut pas être modifié depuis l'interface.

BF-ADM-15 — Interface bilingue et thème

Description : conserver les fonctionnalités du prototype : FR/EN et thème clair/sombre.

Règles :

- Français par défaut.
- Traduction anglaise des libellés.
- Préférence de langue et de thème conservée.
- Aucun texte codé en dur dans les composants.
- Les dates et nombres sont formatés selon la locale.

Critères d'acceptation :

- Le passage en anglais traduit les menus, titres, boutons et messages.
- Le thème sombre reste lisible et contraste suffisamment.
- La préférence survit au rechargement.
- Aucun débordement horizontal en 360 px.

BF-ADM-16 — Sécurité applicative

Description : protéger le back-office.

Règles :

- Toutes les routes `/admin` et `/admin/api/*` exigent une session.
- Vérification du rôle et de la permission côté serveur pour chaque endpoint.

- Requêtes préparées PDO, jamais de concaténation SQL.
- Échappement HTML de toutes les données affichées.
- Protection CSRF sur les écritures.
- En-têtes de sécurité : `Content-Security-Policy`, `X-Content-Type-Options`, `Referrer-Policy`, `Strict-Transport-Security`.
- Limitation de débit sur la connexion et les exports.
- Aucun secret dans le dépôt Git.
- Journalisation sans données sensibles.

Critères d'acceptation :

- Un accès direct à une API d'administration sans session retourne `401`.
- Un accès avec un rôle insuffisant retourne `403`.
- Une requête POST sans CSRF retourne `403`.
- Une injection SQL dans un champ de recherche ne modifie pas la requête.
- Un script `<script>alert(1)</script>` est affiché comme texte.

5. Besoins non fonctionnels

ID	Catégorie	Exigence	Cible mesurable
BNF-01	Performance	Temps de réponse du tableau de bord	p95 < 1,5 s
BNF-02	Performance	Liste paginée	p95 < 1 s pour 100 000 lignes
BNF-03	Performance	Export CSV de 10 000 lignes	< 5 s
BNF-04	Performance	Recherche	p95 < 800 ms
BNF-05	Disponibilité	Disponibilité mensuelle	≥ 99,5 %
BNF-06	Reprise	RTO / RPO	RTO ≤ 4 h / RPO ≤ 24 h
BNF-07	Sécurité	Authentification	Argon2id ou bcrypt, MFA admin
BNF-08	Sécurité	Sessions	Inactivité 30 min, absolue 8 h
BNF-09	Sécurité	Vulnérabilités	0 critique, 0 haute en production
BNF-10	Accessibilité	Conformité	WCAG 2.2 AA
BNF-11	Responsive	Largeurs supportées	360 à 1920 px
BNF-12	Compatibilité	Navigateurs	2 dernières versions Chrome, Firefox, Edge, Safari
BNF-13	Internationalisation	Langues	FR et EN
BNF-14	Maintenabilité	Couverture tests backend	≥ 80 % des services
BNF-15	Maintenabilité	Qualité code	PSR-12, analyse statique sans erreur bloquante
BNF-16	Observabilité	Journalisation	Logs JSON avec identifiant de corrélation
BNF-17	Observabilité	Supervision	Alerte sur erreurs 5xx et échecs de connexion
BNF-18	Sauvegarde	Fréquence	Quotidienne + binlogs
BNF-19	Restauration	Test de restauration	Trimestriel, documenté
BNF-20	Privacy	Données personnelles	Minimisation, durée de conservation définie
BNF-21	UX	Cohérence	Aucun écran blanc, état vide et erreur explicites
BNF-22	Qualité	Console navigateur	0 erreur JavaScript sur les parcours critiques

Détail des exigences clés

Performance

- Le tableau de bord et les listes doivent être paginés côté serveur.
- Les images et icônes doivent être compressées et mises en cache.
- Le rendu initial ne doit pas dépendre d'un appel bloquant à un service externe.
- Les exports volumineux doivent être asynchrones.

Sécurité

- Principe du moindre privilège pour chaque rôle.
- Vérification systématique de l'objet demandé : un administrateur ne doit pas accéder à un objet supprimé ou hors périmètre.
- Protection contre les attaques OWASP Top 10.
- Rotation des secrets et mots de passe.
- MFA pour les comptes administrateurs en production.
- Séparation des comptes applicatifs et des comptes de migration.

Accessibilité

- Navigation clavier complète.
- Focus visible.
- Contraste texte/fond conforme.
- Libellés de formulaires explicites.
- Messages d'erreur reliés aux champs.
- Tableaux avec en-têtes.
- Dialogues avec `role="dialog"`, `aria-modal` et fermeture par `Échap`.

Exploitabilité

- Logs structurés avec niveau, date, acteur, action, identifiant de corrélation.
- Métriques : temps de réponse, taux d'erreur, connexions échouées, exports.
- Mode maintenance documenté.
- Procédure de restauration testée.

6. Contraintes techniques

6.1 Architecture cible

```
Navigateur
  └── Interface admin (HTML/CSS/JS ou framework léger)
        └── API REST /admin/api/* (PHP 8.2)
              ├── Authentification et RBAC
              ├── Services métier
              ├── Validation serveur
              ├── Journal d'audit
              └── MySQL 8.0 (InnoDB, utf8mb4)
```

6.2 Stack recommandée

Couche	Technologie	Contrainte
Serveur web	Apache 2.4 ou Nginx	HTTPS obligatoire en staging/production
Langage	PHP 8.2+	PDO, requêtes préparées, PSR-12
Base	MySQL 8.0+ ou MariaDB 10.11+	InnoDB, utf8mb4, UTC
Frontend	HTML5, CSS3, JavaScript ES2022	Interface responsive, sans dépendance lourde obligatoire
Tests	PHPUnit, Playwright, axe-core	CI obligatoire
Déploiement	Git + CI/CD	Migrations versionnées

6.3 Contraintes issues du prototype

- Le prototype actuel est un fichier HTML autonome avec JavaScript inline et données dans `localStorage`.
- Les données de démonstration sont initialisées par un objet `seed`.
- Les rôles sont simulés : « Aucune permission réelle n'est accordée ».

- Les actions sont enregistrées localement sur l'appareil uniquement.
- La carte est schématique, sans coordonnées GPS réelles.
- L'export CSV est déjà présent et doit être conservé.
- Les identifiants publics (`B001`, `A001`, `T001`, `Z001`, `U001`, `P001`, `S001`, `AV001`) doivent être conservés comme codes fonctionnels.
- Les libellés FR/EN et le thème clair/sombre doivent être conservés.

6.4 Contraintes de développement

- Toute règle métier doit être validée côté serveur, même si elle est aussi validée côté client.
- Toute écriture passe par une transaction si elle touche plusieurs tables.
- Les migrations doivent être réversibles ou accompagnées d'une procédure de retour arrière.
- Les secrets sont lus depuis l'environnement, jamais commités.
- Les noms de tables et colonnes sont en `snake_case`.
- Les dates sont stockées en UTC.
- Les montants et points sont des entiers ; aucun flottant pour les points.
- Les suppressions sensibles sont interdites ou remplacées par un statut.
- Les API retournent du JSON avec des codes HTTP cohérents.
- Les messages d'erreur ne doivent pas exposer de détails SQL.

6.5 Contraintes de sécurité

- Mots de passe : minimum 12 caractères, complexité recommandée, comparaison via `password_verify`.
- Cookies : `HttpOnly`, `Secure`, `SameSite=Lax`.
- CSRF : jeton par session, vérifié sur toute écriture.
- Rate limiting : 5 tentatives de connexion par 15 minutes par compte/IP.
- MFA : TOTP pour les administrateurs.
- Audit immuable côté application.
- Sauvegardes chiffrées.
- Accès base restreint par réseau et utilisateur dédié.

6.6 Contraintes de données

- Encodage `utf8mb4` et collation `utf8mb4_0900_ai_ci`.
- Moteur InnoDB.
- Clés étrangères activées.
- Index sur les colonnes de recherche et de filtre.
- Aucune donnée de production dans les environnements de développement ou de test.
- Jeu de données de démonstration séparé.

7. Environnements et bases de données

7.1 Environnements

Environnement	Usage	Base de données	Données	Accès
Local	Développement	`mobis_admin_dev`	Jeu de démonstration	Développeur
Test / CI	Tests automatisés	`mobis_admin_test`	Fixtures éphémères	CI uniquement
Staging	Recette et validation	`mobis_admin_staging`	Copie anonymisée	Équipe projet
Production	Exploitation	`mobis_admin_prod`	Données réelles	Exploitant, accès restreint

7.2 Règles par environnement

Local

- MySQL 8.0 local ou Docker.
- Base recréée par migrations.
- Jeu de données de démonstration.
- Debug activé, mais jamais accessible publiquement.
- Emails sortants désactivés ou redirigés vers une boîte locale.

Test / CI

- Base MySQL éphémère créée par le pipeline.
- Migrations exécutées à chaque build.
- Fixtures minimales et déterministes.
- Nettoyage après exécution.
- Aucun accès réseau à la production.
- Tests Playwright sur un serveur local.

Staging

- Copie anonymisée de la production ou jeu de données réaliste.
- HTTPS obligatoire.
- Debug désactivé.
- MFA testable.
- Accès protégé par VPN ou liste blanche.
- Restauration de sauvegarde testée avant mise en production.

Production

- MySQL 8.0+ managé ou serveur dédié.
- Sauvegarde quotidienne + binlogs.
- Réplication optionnelle.
- Accès réseau restreint.
- Utilisateur applicatif avec droits DML uniquement.
- Utilisateur de migration séparé avec droits DDL.
- Aucun accès direct au serveur depuis Internet.
- Supervision et alertes actives.

7.3 Variables d’environnement

```
APP_ENV=production
APP_URL=https://bus-maurice.example
APP_DEBUG=false

DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=mobis_admin_prod
DB_USER=mobis_app
DB_PASSWORD=********
DB_CHARSET=utf8mb4

SESSION_NAME=mobis_admin
SESSION_LIFETIME=1800
SESSION_ABSOLUTE_LIFETIME=28800
SESSION_SECURE=true
SESSION_SAMESITE=Lax

MFA_REQUIRED_FOR_ADMIN=true
```

```
AUDIT_RETENTION_MONTHS=12
EXPORT_MAX_ROWS=10000
```

7.4 Rôles MySQL

Rôle	Droits	Usage
mobis_app	SELECT, INSERT, UPDATE, DELETE	Application
mobis_migrator	ALTER, CREATE, DROP, INDEX	Migrations
mobis_readonly	SELECT	Analyse et support
mobis_backup	SELECT, LOCK TABLES, RELOAD	Sauvegarde

7.5 Sauvegarde et restauration

- Sauvegarde complète quotidienne.
- Sauvegarde des binlogs toutes les 5 minutes ou en continu.
- Rétention : 30 jours pour les sauvegardes complètes, 7 jours pour les binlogs.
- Test de restauration trimestriel.
- Documentation de la procédure de restauration.
- Chiffrement des sauvegardes au repos.

8. Modèle de données

8.1 Principes

- Une table par entité métier.
- Clés primaires numériques.
- Codes publics lisibles pour l'interface.
- Clés étrangères explicites.
- Contraintes d'unicité et de validation en base.
- Journal d'audit séparé.
- Aucune suppression physique des données sensibles sans décision explicite.

8.2 Diagramme entité-relation simplifié

```
erDiagram
  roles ||--o{ users : attribue
  roles ||--o{ role_permissions : possede
  permissions ||--o{ role_permissions : accorde
  users ||--o{ admin_sessions : ouvre
  users ||--o{ reports : redige
  users ||--o{ reviews : publie
  users ||--o{ points_transactions : recoit
  users ||--o{ audit_logs : realise
  buses ||--o{ reports : concerne
  routes ||--o{ route_stops : contient
  stops ||--o{ route_stops : figure
  routes ||--o{ reviews : evalue
  buses ||--o{ reviews : evalue
  users ||--o{ reviews : modere
```

8.3 Dictionnaire des tables

Table	Rôle
roles	Rôles applicatifs
permissions	Permissions unitaires
role_permissions	Association rôle/permission
users	Comptes voyageurs et staff
admin_sessions	Sessions actives du back-office

Table	Rôle
auth_attempts	Tentatives de connexion et limitation
buses	Flotte de bus
stops	Arrêts du réseau
routes	Trajets
route_stops	Ordre des arrêts d'un trajet
zones	Zones géographiques
reports	Signalements
reviews	Avis
points_transactions	Historique des ajustements de points
audit_logs	Journal d'audit
settings	Paramètres applicatifs

8.7 Points d'attention sur les données

- Les codes publics doivent rester uniques et stables.
- Les e-mails sont uniques et comparés sans tenir compte de la casse.
- Les coordonnées sont stockées en décimal, pas en texte.
- Les polygones sont stockés en JSON dans cette version ; une évolution vers `GEOMETRY/SRID 4326` est prévue.
- Les suppressions de bus, arrêts, trajets et utilisateurs doivent être encadrées par des règles métier.
- Le solde de points doit toujours correspondre à la somme des transactions. Un contrôle périodique est recommandé.
- Les sessions et jetons CSRF sont stockés sous forme de hash ou de valeur aléatoire, jamais en clair dans les logs.

9. API d'administration

9.1 Principes

- Préfixe : `/admin/api`.
- Réponses JSON.
- Codes HTTP : `200`, `201`, `204`, `400`, `401`, `403`, `404`, `409`, `422`, `429`, `500`.
- Authentification par session.
- CSRF sur toute écriture.
- Permission vérifiée avant chaque action.
- Pagination et filtres via query string.

9.2 Endpoints

Méthode	Endpoint	Permission	Description
POST	/admin/api/login	Public	Connexion
POST	/admin/api/logout	Authentifié	Déconnexion
GET	/admin/api/me	Authentifié	Profil et permissions
GET	/admin/api/dashboard	dashboard.read	Indicateurs
GET	/admin/api/buses	buses.read	Liste des bus
POST	/admin/api/buses	buses.write	Créer un bus
PUT	/admin/api/buses/{id}	buses.write	Modifier un bus
DELETE	/admin/api/buses/{id}	buses.write	Supprimer un bus
GET	/admin/api/stops	stops.read	Liste des arrêts
POST	/admin/api/stops	stops.write	Créer un arrêt
PUT	/admin/api/stops/{id}	stops.write	Modifier un arrêt

Méthode	Endpoint	Permission	Description
DELETE	`/admin/api/stops/{id}`	`stops.write`	Supprimer un arrêt
GET	`/admin/api/routes`	`routes.read`	Liste des trajets
POST	`/admin/api/routes`	`routes.write`	Créer un trajet
PUT	`/admin/api/routes/{id}`	`routes.write`	Modifier un trajet
DELETE	`/admin/api/routes/{id}`	`routes.write`	Supprimer un trajet
GET	`/admin/api/zones`	`zones.read`	Liste des zones
POST	`/admin/api/zones`	`zones.write`	Créer une zone
PUT	`/admin/api/zones/{id}`	`zones.write`	Modifier une zone
DELETE	`/admin/api/zones/{id}`	`zones.write`	Supprimer une zone
GET	`/admin/api/staff`	`staff.read`	Liste du personnel
POST	`/admin/api/staff`	`staff.write`	Créer un membre
PUT	`/admin/api/staff/{id}`	`staff.write`	Modifier un membre
DELETE	`/admin/api/staff/{id}`	`staff.write`	Désactiver un membre
GET	`/admin/api/users`	`users.read`	Liste des voyageurs
PUT	`/admin/api/users/{id}/status`	`users.write`	Suspendre/réactiver
GET	`/admin/api/reports`	`reports.read`	Liste des signalements
PUT	`/admin/api/reports/{id}`	`reports.review`	Traiter un signalement
GET	`/admin/api/reviews`	`reviews.read`	Liste des avis
PUT	`/admin/api/reviews/{id}`	`reviews.moderate`	Modérer un avis
GET	`/admin/api/points`	`points.read`	Soldes et historique
POST	`/admin/api/points/{userId}/adjust`	`points.adjust`	Ajuster les points
GET	`/admin/api/audit`	`audit.read`	Journal d'audit
GET	`/admin/api/export/{module}.csv`	`exports.run`	Export CSV

9.3 Exemple de réponse d'erreur

```
{
  "error": "forbidden",
  "message": "Accès administrateur interdit.",
  "request_id": "req_01J..."
}
```

10. Critères d'acceptation

10.1 Critères globaux

ID	Critère	Vérification
AC-01	Aucun accès admin sans authentification	Test E2E et test API `401`
AC-02	Chaque rôle ne voit que ses modules	Matrice de permissions testée
AC-03	Toute écriture exige un CSRF valide	Test API `403` sans jeton
AC-04	Les données sont persistées en base	Redémarrage et relecture
AC-05	Les contraintes d'unicité sont appliquées	Tests base sur doublons
AC-06	Les actions sensibles sont auditées	Vérification de `audit_logs`
AC-07	L'interface est utilisable en 360 px	Test Playwright responsive
AC-08	L'interface est accessible au clavier	Test manuel et axe-core
AC-09	Les exports CSV sont sûrs	Test d'injection de formule
AC-10	Les points sont cohérents	Test transactionnel

10.2 Critères détaillés par module

Authentification

- **Étant donné** un visiteur non connecté, **quand** il ouvre `/admin`, **alors** il est redirigé vers `/admin/login`.
- **Étant donné** un compte actif, **quand** il saisit les bons identifiants, **alors** il accède au tableau de bord.
- **Étant donné** un compte actif, **quand** il saisit un mauvais mot de passe, **alors** le message reste générique et la tentative est journalisée.
- **Étant donné** 5 échecs consécutifs, **quand** une nouvelle tentative arrive, **alors** elle est bloquée 15 minutes.
- **Étant donné** une session inactive depuis 30 minutes, **quand** l'utilisateur agit, **alors** il doit se reconnecter.

Tableau de bord

- **Étant donné** des données en base, **quand** le tableau de bord s'affiche, **alors** les compteurs correspondent aux requêtes SQL.
- **Étant donné** 4 bus, **quand** le tableau de bord s'affiche, **alors** les 4 bus sont listés.
- **Étant donné** un signalement en attente, **quand** le tableau de bord s'affiche, **alors** il apparaît dans « À votre attention ».

Bus

- **Étant donné** une plaque existante, **quand** l'utilisateur crée un bus avec la même plaque, **alors** une erreur `409` est affichée.
- **Étant donné** un bus actif, **quand** il passe en panne, **alors** le compteur des bus en service diminue.
- **Étant donné** un opérateur, **quand** il tente une suppression, **alors** l'action est autorisée si sa permission le permet et auditée.

Arrêts

- **Étant donné** une latitude de 100, **quand** l'utilisateur enregistre, **alors** une erreur de validation est affichée.
- **Étant donné** un arrêt utilisé par un trajet, **quand** l'utilisateur le supprime, **alors** la suppression est refusée.
- **Étant donné** un arrêt inactif, **quand** l'utilisateur crée un trajet, **alors** l'arrêt n'est pas proposé.

Trajets

- **Étant donné** un trajet avec un seul arrêt, **quand** l'utilisateur enregistre, **alors** une erreur est affichée.
- **Étant donné** trois arrêts, **quand** l'utilisateur change l'ordre, **alors** l'ordre est persisté.
- **Étant donné** un trajet supprimé, **quand** l'utilisateur consulte les arrêts, **alors** les arrêts existent toujours.

Zones

- **Étant donné** deux sommets, **quand** l'utilisateur enregistre, **alors** une erreur est affichée.
- **Étant donné** trois sommets, **quand** l'utilisateur enregistre, **alors** la zone est créée.
- **Étant donné** une zone inactive, **quand** l'interface voyageur s'affiche, **alors** la zone n'est pas visible.

Personnel et utilisateurs

- **Étant donné** le dernier administrateur, **quand** l'utilisateur tente de le désactiver, **alors** l'action est refusée.
- **Étant donné** un voyageur suspendu, **quand** il tente de se connecter, **alors** l'accès est refusé.
- **Étant donné** un opérateur, **quand** il ouvre `/admin/staff`, **alors** il reçoit `403`.

Signalements et avis

- **Étant donné** un signalement `pending`, **quand** l'utilisateur le confirme, **alors** le statut passe à `confirmed` et `handled_by` est renseigné.
- **Étant donné** un signalement confirmé, **quand** l'utilisateur consulte le bus, **alors** l'état du bus n'a pas changé automatiquement.

- **Étant donné** un avis `pending`, **quand** le modérateur le publie, **alors** il devient visible côté voyageur.
- **Étant donné** un avis `hidden`, **quand** l'interface voyageur s'affiche, **alors** l'avis n'est pas visible.

Points

- **Étant donné** un solde de 5, **quand** l'admin ajoute 10, **alors** le solde devient 15.
- **Étant donné** un solde de 5, **quand** l'admin retire 10, **alors** l'opération est refusée.
- **Étant donné** un motif vide, **quand** l'admin valide, **alors** une erreur est affichée.
- **Étant donné** un double clic avec la même clé d'idempotence, **quand** les deux requêtes arrivent, **alors** une seule transaction est enregistrée.

Export

- **Étant donné** un filtre actif, **quand** l'utilisateur exporte, **alors** le CSV contient uniquement les lignes filtrées.
- **Étant donné** une valeur commençant par `=`, **quand** elle est exportée, **alors** elle est préfixée pour empêcher l'exécution d'une formule.
- **Étant donné** un export de plus de 10 000 lignes, **quand** l'utilisateur le demande, **alors** un traitement asynchrone est proposé.

Audit

- **Étant donné** une modification de bus, **quand** l'action est terminée, **alors** une ligne d'audit est créée.
- **Étant donné** un opérateur, **quand** il tente d'ouvrir le journal, **alors** l'accès est refusé.
- **Étant donné** une ligne d'audit, **quand** l'utilisateur tente de la modifier, **alors** l'interface ne le permet pas.

11. Définition du test fini (Definition of Done)

11.1 Pour une user story

Une story est « done » lorsque :

- Les critères d'acceptation sont couverts par au moins un test automatisé ou une recette documentée.
- Le code est relu et approuvé par au moins une autre personne.
- Les tests unitaires et d'intégration passent.
- Le test E2E Playwright du parcours concerné passe.
- Les règles métier sont validées côté serveur.
- Les messages d'erreur sont traduits FR/EN.
- L'accessibilité de base est vérifiée : clavier, focus, contraste, libellés.
- Le responsive est vérifié en 360, 768 et 1440 px.
- Les actions sensibles sont auditées.
- La documentation API ou utilisateur est mise à jour si nécessaire.
- Aucun secret ni donnée personnelle n'est commité.
- La CI est verte.
- Le déploiement en staging est effectué et validé.

11.2 Pour une release

La release est « done » lorsque :

- 100 % des critères d'acceptation Must sont validés.
- 0 anomalie critique ou haute ouverte.
- Les tests de non-régression passent.

- Le rapport de tests Playwright est généré et archivé.
- Les tests de sécurité OWASP sont passés.
- Les tests d’accessibilité ne remontent aucune erreur critique ou sérieuse.
- Les performances respectent les budgets définis.
- Les migrations ont été testées en avant et en retour.
- La sauvegarde et la restauration ont été vérifiées.
- Le monitoring et les alertes sont actifs.
- La documentation d’exploitation est à jour.
- La validation du product owner est obtenue.
- La release est déployée en production avec un plan de retour arrière.

12. Stratégie de tests et recette

12.1 Tests unitaires

- Validation des champs.
- Matrice des permissions.
- Règles de points.
- Calcul des indicateurs du tableau de bord.
- Formatage CSV.
- Génération et vérification CSRF.

12.2 Tests d’intégration

- Contraintes de clés étrangères.
- Unicité des plaques, e-mails et codes.
- Transactions de points.
- Audit automatique.
- Suppression protégée des arrêts utilisés.
- Sessions et expiration.

12.3 Tests E2E Playwright

- Connexion, déconnexion, redirection.
- Accès refusé par rôle.
- CRUD bus, arrêts, trajets, zones.
- Gestion du personnel et des utilisateurs.
- Traitement d’un signalement.
- Modération d’un avis.
- Ajustement de points.
- Export CSV.
- Responsive 360/390/768/1440.
- Absence d’erreur JavaScript.

12.4 Tests de sécurité

- Injection SQL.
- XSS stocké et réfléchi.
- CSRF.

- IDOR (accès à l'objet d'un autre utilisateur).
- Escalade de privilèges.
- Force brute et verrouillage.
- Fixation de session.
- Exposition de données sensibles.

12.5 Tests de performance

- 50 utilisateurs admin simultanés.
- 100 000 utilisateurs.
- 10 000 signalements.
- Export de 10 000 lignes.
- Mesure p95 des endpoints principaux.

12.6 Recette utilisateur

- Scénario administrateur complet.
- Scénario opérateur.
- Scénario modérateur.
- Vérification des libellés FR/EN.
- Vérification du thème sombre.
- Vérification des messages d'erreur.

13. Livrables

- Ce cahier des charges.
- Schéma SQL et migrations.
- Jeu de données de démonstration.
- API d'administration documentée.
- Interface admin responsive et accessible.
- Tests unitaires, d'intégration et E2E.
- Rapport de tests Playwright HTML/JSON.
- Documentation d'exploitation et de restauration.
- Guide utilisateur administrateur.
- Procédure de déploiement.

14. Risques et points ouverts

Risque / point ouvert	Impact	Mesure proposée
Prototype sans backend réel	Migration importante	Développer l'API et les migrations avant l'interface finale
Rôles simulés dans le prototype	Confusion des droits	Valider la matrice RBAC avec le métier
Barème des points non défini	Règles instables	Limiter la v1 aux ajustements manuels
Carte schématique non géographique	Écart avec le réel	Prévoir une évolution vers un fournisseur cartographique
Données personnelles	Conformité	Politique de conservation et anonymisation
Absence de MFA dans le prototype	Risque de compromission	Imposer la MFA aux administrateurs
URL `/admin/objectif` en 404	Référence incomplète	Utiliser `/admin` comme source de vérité
Suppression de données	Perte ou incohérence	Remplacer par des statuts et auditer
Volume des exports	Temps de réponse	Limite synchrone et export asynchrone

15. Annexes

Annexe A — Correspondance prototype / base

Prototype	Table cible	Remarque
db.buses	buses	id devient code
db.stops	stops	Coordonnées en décimal
db.routes	routes + route_stops	Les arrêts deviennent des lignes ordonnées
db.zones	zones	points devient points_json
db.users	users	Rôle traveller
db.staff	users	Rôles operator, moderator, admin
db.reports	reports	bus devient bus_id
db.reviews	reviews	Ajout de route_id/bus_id
db.pointlog	points_transactions	Ajout de balance_after et performed_by
db.activity	audit_logs	Journal persistant

Annexe B — Écrans du back-office

Écran	Route cible	Permission
Connexion	/admin/login	Public
Tableau de bord	/admin/dashboard	dashboard.read
Bus	/admin/buses	buses.read
Arrêts	/admin/stops	stops.read
Trajets	/admin/routes	routes.read
Zones	/admin/zones	zones.read
Personnel	/admin/staff	staff.read
Utilisateurs	/admin/users	users.read
Signalements	/admin/reports	reports.read
Avis	/admin/reviews	reviews.read
Points	/admin/points	points.read
Audit	/admin/audit	audit.read

Annexe C — Règles de validation transverses

- Chaînes : `trim`, longueur maximale, rejet des caractères de contrôle.
- E-mail : format RFC et longueur maximale 254.
- Téléphone : format international recommandé.
- Plaque : normalisation en majuscules.
- Codes publics : `^[A-Z][0-9]{3,}$`.
- Points : entier, jamais de décimale.
- Dates : format ISO 8601 en entrée, UTC en base.
- JSON : validation de structure côté serveur.
- Fichiers : aucun upload dans le périmètre admin v1.
- Suppression : confirmation obligatoire et audit.